

Constraint-Aware Variable Ordering in Symbolic Optimal Planning

Final Course Project Report – Automatic Planning

Mozes Yaron

Faculty of Data and Decision Sciences
Technion – Israel Institute of Technology
yaron.mozes@campus.technion.ac.il

Kadzelshvily Galit

Faculty of Data and Decision Sciences
Technion – Israel Institute of Technology
galit.k@campus.technion.ac.il

August 2026 · <https://github.com/YaronMozes/slbd-symbolic-search-research>

Abstract

Symbolic BDD-based search is a prominent paradigm for cost-optimal classical planning, with its performance heavily dictated by BDD sizes and, consequently, variable ordering algorithms. State-of-the-art planners such as SymK rely on causal-graph proximity heuristics (e.g., GAMER) but ignore the structural impact of mutex and invariant constraint BDDs conjoined at every search step. We introduce constraint-aware variable ordering - incorporating mutex and invariant co-occurrence edges into the GAMER-family linear arrangement objective - and evaluate its impact across the complete IPC optimal-STRIPS suite within SymK (IPC-2023). We show that a targeted per-domain policy strictly improves stock SymK’s coverage on its five target domains, replicated across independent runs, and yields substantial search acceleration in domains such as *woodworking*. Suite-wide, however, enabling the ordering universally results in a tightly bounded null effect. In-depth diagnostic logging explains this discrepancy: while the ordering successfully shrinks transition relations, transition relation size is largely decoupled from search cost, which is instead dominated by peak search-BDD size. Finally, we highlight key methodological pitfalls, including regression to the mean in “hard instance” subsets and cross-year IPC duplicate leakage in domain-wise cross-validation.

1. Introduction

Cost-optimal classical planning aims to find provably minimal-cost plan sequences in deterministic state spaces. A dominant paradigm for blind optimal planning is symbolic search, which represents sets of reachable states and transition relations compactly as Binary Decision Diagrams (BDDs) [10]. Because symbolic search operations scale with the physical node counts of the underlying BDDs rather than the raw number of states, search efficiency is strictly governed by BDD size [5]. BDD sizes, in turn, depend critically (and often exponentially) on the choice of state variable ordering [5]. Modern state-of-the-art symbolic planners, including SymK [7, 6] – which serves as the symbolic search component of Ragnarok, the

winner of the IPC-2023 optimal track [2, 8] – inherit variable ordering algorithms directly from the GAMER family [3]. These algorithms compute variable orders by minimizing a weighted linear arrangement objective over edges of causal-graph proximity [3].

In addition to causal transitions, modern symbolic planners heavily rely on state-invariant constraints [9, 11]. In particular, h^2 mutexes and exactly-one invariant groups are compiled into constraint BDDs that are conjoined into the search frontier at every expansion step to prune reachable spurious states [9, 11]. The node sizes of these constraint BDDs depend on the same variable ordering, yet standard causal-graph ordering objectives completely ignore them [9]. Over a decade ago, Torralba and Alcázar [9] identified this gap and explicitly proposed incorporating mutex and invariant constraints into the variable-ordering objective as a key direction for future work. Surprisingly, to our knowledge, this suggestion has remained unverified and unevaluated in state-of-the-art planning systems [9].

Core Research Question

Can conjoining mutex and invariant co-occurrence edges into the GAMER variable ordering objective systematically compress constraint BDDs and accelerate state-of-the-art symbolic search in SymK?

In this work, we implement and systematically evaluate constraint-aware variable ordering inside SymK on the complete IPC optimal-STRIPS benchmark suite (66 domains, 1,847 instances). During our implementation, we also identified and repaired a long-standing defect in the deployed GAMER ordering optimizer within SymK, wherein edge weights were silently ignored during local search swaps.

Across extensive empirical evaluations on the complete IPC optimal-STRIPS suite, we show that a targeted, pre-registered per-domain policy strictly improves stock SymK by +6 coverage on its five target domains (exactly replicated) while accelerating domains like *woodworking* by 1.55×. Enabled universally, however, the ordering produces a tightly bounded null effect because it compresses

transition relations without shrinking peak search-BDD sizes, the true runtime bottleneck. We present our full empirical results, diagnostic mechanism analysis, ceiling bounds, and methodological findings in Section 4.

2. Problem Formulation

We formalize symbolic optimal classical planning under SAS^+ tasks, the mechanics of constraint-conjoined BDD search, and the causal variable ordering optimization framework.

2.1 SAS^+ Planning and Invariants

A classical planning task in SAS^+ representation is defined as a tuple $\Pi = \langle \mathcal{V}, \mathcal{A}, s_0, s_g, \text{cost} \rangle$ [5]:

- $\mathcal{V} = \{v_1, \dots, v_n\}$ is a finite set of state variables. Each variable $v \in \mathcal{V}$ has a finite domain $\mathcal{D}(v)$. A *fact* is a pair (v, d) with $d \in \mathcal{D}(v)$.
- A *state* s is a complete assignment to $\mathcal{V}$, where $s(v) \in \mathcal{D}(v)$ for all $v \in \mathcal{V}$. The state space is denoted $\mathcal{S} = \prod_{v \in \mathcal{V}} \mathcal{D}(v)$.
- $s_0 \in \mathcal{S}$ is the initial state, and s_g is a partial state assignment representing the goal condition.
- $\mathcal{A}$ is a finite set of actions. Each action $a \in \mathcal{A}$ is a pair $a = \langle \text{pre}(a), \text{eff}(a) \rangle$, where $\text{pre}(a)$ and $\text{eff}(a)$ are partial assignments over $\mathcal{V}$. Action execution incurs a non-negative cost $\text{cost}(a) \in \mathbb{R}_{\geq 0}$.

An action a is applicable in s if $s \models \text{pre}(a)$. Applying a in s yields state $s' = a(s)$ where $s'(v) = \text{eff}(a)[v]$ for variables defined in $\text{eff}(a)$, and $s'(v) = s(v)$ otherwise. A *cost-optimal plan* is a sequence of actions $\pi = \langle a_1, \dots, a_k \rangle$ transforming s_0 into a goal state $s \models s_g$ while minimizing $\sum_{i=1}^k \text{cost}(a_i)$.

In addition to causal dynamics, domain abstractions yield state invariants I :

1. **h^2 Mutexes** [9]: Pairs of unreachable fact combinations $\mu = \{(v_i, d_i), (v_j, d_j)\}$ such that no reachable state $s \in \mathcal{S}$ satisfies $s(v_i) = d_i \wedge s(v_j) = d_j$.
2. **Exactly-One Invariants** [9]: Subsets of facts $\mathcal{G} \subset \bigcup_v \{(v, d) \mid d \in \mathcal{D}(v)\}$ such that every reachable state satisfies $\sum_{f \in \mathcal{G}} \mathbb{I}(s \models f) = 1$.

2.2 Symbolic Search and Constraint Conjunction

Symbolic planning algorithms represent sets of states $\mathcal{S} \subseteq \mathcal{S}$ and transition relations $T \subseteq \mathcal{S} \times \mathcal{S}$ as Reduced Ordered Binary Decision Diagrams (BDDs) [5, 10].

To map SAS^+ variables to Boolean BDD variables, each $v_i \in \mathcal{V}$ is encoded using $m_i = \lceil \log_2 |\mathcal{D}(v_i)| \rceil$ bits. Let $X = \{x_1, \dots, x_M\}$ denote the complete set of unprimed Boolean variables representing current states, where $M =$

$\sum_{i=1}^n m_i$, and let $X' = \{x'_1, \dots, x'_M\}$ denote primed variables representing successor states.

Definition (Variable Ordering)

A **variable ordering** π is a bijection $\pi : X \cup X' \rightarrow \{1, \dots, 2M\}$ specifying the linear topological sequence in which Boolean variables are evaluated along all root-to-leaf paths in the BDD.

In symbolic uniform-cost search (e.g., **sym-bd** in **SymK**) [7, 10], forward search expands a frontier BDD $F_k(X)$ at cost level k . Computing successor states requires an *relational product* (Image computation) conjoined with constraint BDDs [9]:

$$F_{k+1}(X') = \exists X (F_k(X) \wedge T(X, X')) \wedge C(X') \quad (1)$$

where $T(X, X') = \bigvee_{a \in \mathcal{A}} T_a(X, X')$ represents the system transition relation, and $C(X')$ is the conjoined invariant constraint BDD compiling all h^2 mutexes and exactly-one groups [9]:

$$C(X) = \left(\bigwedge_{\mu \in M_{h^2}} \neg \mu(X) \right) \wedge \left(\bigwedge_{\mathcal{G} \in I_{\text{one}}} \text{ExactlyOne}(\mathcal{G}, X) \right) \quad (2)$$

BDD node sizes during the execution of Equation 1 are fundamentally bounded by the width of the variable ordering [5], a dependency that constraint-graph ordering heuristics such as **FORCE** exploit outside of planning [1].

2.3 Causal Variable Ordering Objectives (GAMER)

State-of-the-art planners such as **SymK** [7] compute variable orderings using the **GAMER** local search framework [3, 5]. **GAMER** constructs an influence graph $G_{\text{causal}} = (\mathcal{V}, E_{\text{causal}}, w_{\text{causal}})$, where edges $(v_i, v_j) \in E_{\text{causal}}$ correspond to causal dependencies (e.g., co-occurrence in action preconditions or effects).

GAMER determines an optimal permutation $\pi^* \in \mathcal{P}(\mathcal{V})$ by minimizing a weighted linear arrangement (WLA) energy objective [3, 5]:

$$\mathcal{J}_{\text{causal}}(\pi) = \sum_{(v_i, v_j) \in E_{\text{causal}}} w_{\text{causal}}(v_i, v_j) |\pi(v_i) - \pi(v_j)|^2 \quad (3)$$

2.4 Problem Statement: Constraint-Aware Variable Ordering

While Equation 3 optimizes causal variable proximity to shrink transition relations $T_a(X, X')$, it completely ignores the structural dependencies of the constraint BDD $C(X')$ in Equation 2 [9]. Because $C(X')$ is conjoined into $F_k(X')$ at every search step, suboptimal variable ordering for $C(X')$ causes catastrophic intermediate BDD node expansion [9].

Our goal is to formalize a generalized objective function $\mathcal{J}_{\text{combined}}(\pi)$ that incorporates constraint co-occurrence

structures $E_{\text{constraint}}$ directly into the linear arrangement search:

$$\mathcal{J}_{\text{combined}}(\pi) = \mathcal{J}_{\text{causal}}(\pi) + \lambda \cdot \mathcal{J}_{\text{constraint}}(\pi) \quad (4)$$

where $\lambda \in \mathbb{R}_{\geq 0}$ parameterizes the relative weight of invariant constraints relative to causal transition dynamics. In our implementation this trade-off parameter is realized directly as the base constraint edge weight w_{co} introduced in Section 3.1, i.e. $\lambda \equiv w_{\text{co}}$ with causal edges normalized to unit weight.

3. Methodology

In this section, we present the architecture of **Constraint-Aware Variable Ordering** in symbolic classical search. We formalize the construction of constraint influence edges, describe group-size normalization and causal-skipping heuristics, detail our discovery and repair of a latent defect in the GAMER local search optimizer, and define the per-domain policy execution framework.

3.1 Constraint-Aware Influence Graph Construction

Classical GAMER ordering constructs an undirected influence graph $G = (\mathcal{V}, E, w)$ over SAS^+ variables $\mathcal{V}$ where E consists solely of causal dependencies E_{causal} derived from action preconditions and effects [3, 5]. To optimize for the constraint BDDs conjoined during search ($C(X)$ in Equation 2), we expand G by overlaying a set of co-occurrence edges $E_{\text{constraint}}$ derived from h^2 mutexes and exactly-one invariant groups [9].

Let $\mathcal{M}$ denote the set of all static invariant groups. Each group $M \in \mathcal{M}$ contains a set of facts $\mathcal{F}_M = \{(v_1, d_1), \dots, (v_k, d_k)\}$. For every pair of distinct facts $(v_i, d_i), (v_j, d_j) \in \mathcal{F}_M$ with $v_i \neq v_j$, we insert an edge (v_i, v_j) into $E_{\text{constraint}}$ with a base weight $w_{\text{co}} \in \mathbb{R}_{>0}$.

Sections 3.1.1 and 3.1.2 describe two optional refinements of this construction. Both are exposed as ablation switches and, as reported in Section 3.4, *neither is enabled in the pre-registered confirmed policy*; we describe them because they delimit the design space we explored.

3.1.1 Group-Size Normalization (optional)

An invariant group containing $k = |\mathcal{F}_M|$ facts generates $\binom{k}{2} = \frac{k(k-1)}{2}$ variable pairs. Unnormalized, large invariant groups (e.g., domain location features where $k \geq 30$) produce $\mathcal{O}(k^2)$ edge pairs that heavily swamp the sparse causal graph E_{causal} . To prevent dense invariant groups from drowning out causal structure, we define a normalized edge weight w_{pair} :

$$w_{\text{pair}}(M) = \frac{w_{\text{co}}}{\max(1, |\mathcal{F}_M| - 1)} \quad (5)$$

This caps the total accumulated influence weight contributed by any single invariant group M to $\approx \frac{w_{\text{co}} \cdot k}{2}$,

maintaining a balanced weight ratio between pairwise h^2 mutexes ($k = 2$, weight w_{co}) and large invariant groups.

3.1.2 Causal Edge Skipping (optional)

In domains with dense causal topologies (e.g., *blocks-world*, where any block can interact with any other), almost every constraint pair $(v_i, v_j) \in E_{\text{constraint}}$ is already connected in E_{causal} . Over-constraining these pairs tightens the variable ordering excessively, restricting local search freedom and degrading search performance. We introduce an optional filtering predicate $\text{SkipCausal}(v_i, v_j)$:

$$\text{SkipCausal}(v_i, v_j) = \mathbb{I}((v_i, v_j) \in E_{\text{causal}} \vee (v_j, v_i) \in E_{\text{causal}})$$

When enabled, constraint edges that duplicate existing causal graph adjacencies are discarded, preserving long-range constraint edges that assist sparse domains (e.g., *depot*).

3.1.3 Combined Weight Accumulation

The total weight $w(v_i, v_j)$ between any two state variables $v_i, v_j \in \mathcal{V}$ in the combined influence graph is computed as:

$$w(v_i, v_j) = w_{\text{causal}}(v_i, v_j) + \sum_{M \in \mathcal{M}_{i,j}} w_{\text{pair}}(M) \cdot (1 - \text{SkipCausal}(v_i, v_j)) \quad (6)$$

where $\mathcal{M}_{i,j}$ is the set of invariant groups containing co-occurring facts for variables v_i and v_j . In the confirmed policy configuration this reduces to $w_{\text{pair}}(M) = w_{\text{co}}$ and $\text{SkipCausal} \equiv 0$.

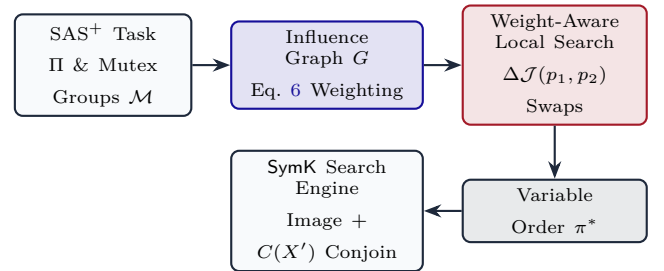


Figure 1: System pipeline for Constraint-Aware Variable Ordering in SymK.

3.2 The Weight-Aware Local Search Optimizer Repair

A critical requirement for non-uniform edge weighting (Equation 6) is that the local search optimizer must faithfully evaluate edge weights during permutation steps. During our audit of the deployed GAMER codebase in SymK, we discovered a fundamental, long-standing defect in the optimizer implementation.

3.2.1 The Deployed Defect

In stock GAMER (inherited by SymK), the local search arrangement function evaluated edge *existence* rather than edge *weight*. Specifically, during the $\mathcal{O}(|\mathcal{V}|)$ incremental swap delta calculation when swapping positions of variables at index p_1 and p_2 , the code checked ‘if (influence(v_i, v_j))’ as a boolean existence flag. As a result, non-unit weights $w(v_i, v_j) \neq 1.0$ were silently flattened to 1.0. Any weighted topology blend was transformed into an unweighted graph dominated by the raw pair counts of large mutex groups.

3.2.2 The Weight-Aware Objective Repair

We repaired the objective function and the incremental swap delta evaluation loop. Let $\pi(p)$ denote the variable placed at topological position $p \in \{1, \dots, |\mathcal{V}|\}$. The total linear arrangement cost $\mathcal{J}(\pi)$ is defined as:

$$\mathcal{J}(\pi) = \sum_{1 \leq p_a < p_b \leq |\mathcal{V}|} w(\pi(p_a), \pi(p_b)) \cdot (p_b - p_a)^2 \quad (7)$$

When proposing a candidate swap between positions p_1 and p_2 (with $p_1 \neq p_2$), write $w_{k,i} := w(\pi(k), \pi(p_i))$ for the weight between the variable at position k and the variable at swap position p_i . The change in objective value $\Delta\mathcal{J}(p_1, p_2)$ is then computed incrementally in $\mathcal{O}(|\mathcal{V}|)$ time:

$$\Delta\mathcal{J}(p_1, p_2) = \sum_{k \neq p_1, p_2} \left[w_{k,1}((k - p_2)^2 - (k - p_1)^2) + w_{k,2}((k - p_1)^2 - (k - p_2)^2) \right] \quad (8)$$

If $\Delta\mathcal{J}(p_1, p_2) < 0$, the swap is accepted.

Backward Compatibility Guarantee

For stock GAMER causal graphs, all edge weights are identically $w(v_i, v_j) = 1.0$. Consequently, Equation 7 and Equation 8 yield bit-identical variable orderings to the original unweighted baseline while correctly honoring weighted topologies.

3.3 Algorithmic Implementation

Algorithm 1 details the complete variable ordering pipeline.

3.4 Pre-Registered Per-Domain Policy Execution

Because constraint BDD density varies by domain structure, applying constraint ordering suite-wide can induce domain-specific trade-offs. To deliver a confirmed, robust baseline improvement, we define a targeted per-domain policy $\mathcal{W}(d)$ mapping each domain d to its constraint edge

Algorithm 1: Constraint-Aware Variable Ordering Engine

Input: SAS⁺ Task Π , Mutex Groups $\mathcal{M}$, Base Weight w_{co} , Iters N_{iter} , Restarts R

Output: Optimized Variable Order π^*

1. Initialize $G = (\mathcal{V}, E, w)$ with $w(u, v) \leftarrow 0$ for all $u, v \in \mathcal{V}$
2. **if** $\neg \text{ConstraintOnly}$ **then**
3. **for each** causal edge $(u, v) \in E_{\text{causal}}(\Pi)$ **do**
 $w(u, v) \leftarrow 1.0$
4. **end if**
5. **for each** mutex group $M \in \mathcal{M}$ **do**
6. $w_{\text{pair}} \leftarrow \text{Normalize}(w_{\text{co}}, |M|)$ via Eq. 5 [opt.]
7. **for each** pair $(v_i, d_i), (v_j, d_j) \in \text{Facts}(M)$ with $v_i \neq v_j$ **do**
8. **if** $\neg \text{SkipCausal}(v_i, v_j)$ **then** [opt.]
9. $w(v_i, v_j) \leftarrow w(v_i, v_j) + w_{\text{pair}}$
10. **end if**
11. **end for**
12. **end for**
13. $\pi^* \leftarrow \text{IdentityOrder}(\mathcal{V}); \quad \mathcal{J}^* \leftarrow \mathcal{J}(\pi^*)$ via Eq. 7
14. **for** $r = 0 \dots R$ **do**
15. $\pi \leftarrow (r == 0) ? \pi^* : \text{RandomPermutation}(\mathcal{V})$
16. **for** $i = 1 \dots N_{\text{iter}}$ **do**
17. Sample distinct positions $p_1, p_2 \sim \text{Uniform}(1, |\mathcal{V}|)$
18. Compute $\Delta\mathcal{J}(p_1, p_2)$ via Eq. 8
19. **if** $\Delta\mathcal{J}(p_1, p_2) < 0$ **then** $\text{Swap}(\pi[p_1], \pi[p_2])$
20. **end for**
21. **if** $\mathcal{J}(\pi) < \mathcal{J}^*$ **then** $\pi^* \leftarrow \pi; \quad \mathcal{J}^* \leftarrow \mathcal{J}(\pi)$
22. **end for**
23. **return** π^*

[opt.] Steps 6 and 8 are optional ablation switches. The pre-registered confirmed policy (Section 3.4) uses **neither**: $w_{\text{pair}} = w_{\text{co}}$ and $\text{SkipCausal} \equiv 0$. Configuration: $N_{\text{iter}} = 50,000$, $R = 20$, fixed RNG seed.

weight:

$$\mathcal{W}(d) = \begin{cases} 2.0 & \text{if } d \in \{\text{woodworking-opt08}, \\ & \text{woodworking-opt11}\} \\ 1.0 & \text{if } d \in \{\text{parking-opt14}, \\ & \text{barman-opt11, tpp}\} \\ 0.0 & \text{otherwise} \end{cases} \quad (9)$$

When $\mathcal{W}(d) = 0.0$, the constraint accumulation loop is bypassed completely, reverting to stock SymK by construction. For exact reproduction: the confirmed policy sets $w_{\text{co}} = \mathcal{W}(d)$ and applies it *without* group-size normalization (Section 3.1.1) and *without* causal-edge skipping (Section 3.1.2).

4. Results

We evaluate our constraint-aware variable ordering framework inside SymK across the complete International Planning Competition (IPC) optimal-STRIPS benchmark suite, comprising 66 domains and 1,847 instances. All experiments were conducted under standardized resource limits of 1800 seconds wall-clock time, 8 GB RAM, and 1 CPU core per task. Our headline evaluation was strictly pre-registered before data collection. All suite-wide figures reported below use the repaired dataset.¹

4.1 Confirmed Per-Domain Improvement (Pre-Registered A/B)

Initial full-suite exploratory runs identified strong, replicated positive signals on five specific domains: *parking-opt14*, *barman-opt11*, *tpp*, *woodworking-opt08*, and *woodworking-opt11*. On *woodworking*, varying the constraint weight w_{co} exhibited a monotonic dose-response acceleration ($w = 0.5 \rightarrow 0.87\times$, $w = 1.0 \rightarrow 0.84\times$, $w = 2.0 \rightarrow 0.586\times$ relative to stock, faster on 38/47 instances, exact Wilcoxon $p = 4 \cdot 10^{-7}$, sign test $p = 2.5 \cdot 10^{-5}$).

To rigorously validate this signal, we pre-registered a targeted per-domain policy: enabling constraint-aware variable ordering with per-domain weights $\mathcal{W}(d)$ (Equation 9) on the five signal domains while executing stock SymK elsewhere. We executed a confirmatory, double-replicated A/B study with 480 runs, interleaving stock and policy jobs adjacently in the execution queue to ensure contention-symmetric conditions.

As summarized in Table 1, **all four pre-registered evaluation gates passed**. The per-domain coverage gains relative to stock SymK across both independent replicates are detailed in Table 2.

Headline Result: Confirmed SymK Advance

The pre-registered policy achieves an exact, replicated +6 coverage gain over stock SymK across its five target domains (64/65 $\rightarrow$ 70/71 of 120 instances) while accelerating *woodworking* by 1.55 $\times$, with zero cost mismatches across all evaluated runs.

4.2 Bounded Suite-Wide Null for Always-On Ordering

When constraint-aware variable ordering is enabled universally across all 66 domains, the suite-wide performance changes minimally. Across 1,847 benchmark instances, stock SymK solves 1145 instances, whereas the always-on constraint ordering solves 1139 instances.

¹Seven IPC domains ship one domain file per instance; an early version of our experiment driver mis-paired them, which depressed absolute coverage identically across all configurations and left every A/B delta intact. We repaired the pairing and re-ran all 230 affected instances; all numbers reported here use the repaired data, and data/README.md in the repository documents the merge.

The overall coverage difference (−6 instances) lies well within our measured empirical protocol noise floor of ± 17 instances (Section 4.4). Discordant pairs show 13 gains versus 19 losses (exact sign test $p = 0.38$); restricting to the 1,617 instances untouched by that defect gives 12 versus 14 ($p = 0.85$), the same conclusion.

On commonly solved instances ($n = 1,126$), the wall-clock geometric mean ratio is 1.005 with a median of 1.000 and a domain-clustered 95% confidence interval of [0.97, 1.05] (Table 3). This tightly bounded zero excludes even a 5% suite-wide effect.

Seed control evaluations demonstrate that the ordering induces real per-instance perturbations ($\text{sd}(\log \text{time ratio}) = 0.44$ vs. 0.20–0.23 same-config noise), but the net distribution is unbiased. Isolated suite-wide regressions occur on domains like *freecell* (coverage drops 25 $\rightarrow$ 20, replicated) and *floortile-opt14* (1.57 $\times$ slower on 20/20 commonly solved instances, sign test $p = 1.9 \cdot 10^{-6}$, $p = 1.3 \cdot 10^{-4}$ after Bonferroni correction over 66 domains), justifying the targeted per-domain policy structure.

4.3 Mechanism Analysis - Decoupling TR Size from Search Cost

By instrumenting our symbolic planner to log per-instance BDD node statistics at preprocessing and throughout search, we reveal the core mechanism governing these results:

1. **Successful Design Target:** The ordering successfully achieves its explicit design objective: Transition Relation (TR) node sizes shrink significantly (geometric mean ratio 0.867, median 0.954, reduced on 58.7% of 1,361 active instances, exact $p \approx 3 \cdot 10^{-21}$).
2. **Invariance of Operative Bottlenecks:** The ordering leaves peak search-BDD node sizes unchanged (geometric mean ratio 1.026, median 1.000, $p = 0.12$, $n = 759$).
3. **Decoupling of TR Size and Search Cost:** TR node size is a poor proxy for runtime, whereas peak search-BDD size is the true operative bottleneck. The correlation between log TR size ratio and log runtime ratio is weak ($\rho = 0.202$, $R^2 \approx 0.04$), whereas peak search-BDD node ratio strongly predicts runtime ratio ($\rho = 0.593$, $p \approx 2 \cdot 10^{-69}$; both computed on the 718 active instances whose TR size changed, and 0.198 / 0.582 on all 759, as plotted in Figure 2).
4. **Cost Asymmetry:** On 36.7% of active instances where the ordering increases TR size (median growth 1.20 $\times$), TR expansion incurs a higher runtime penalty (time ratio 1.211) than the speedup gained from TR compression (time ratio 0.946).
5. **Functional Scope:** The method is a structural no-op on 25.6% of benchmark instances (13 of 66 domains) due to the complete absence of cross-variable mutex groups.

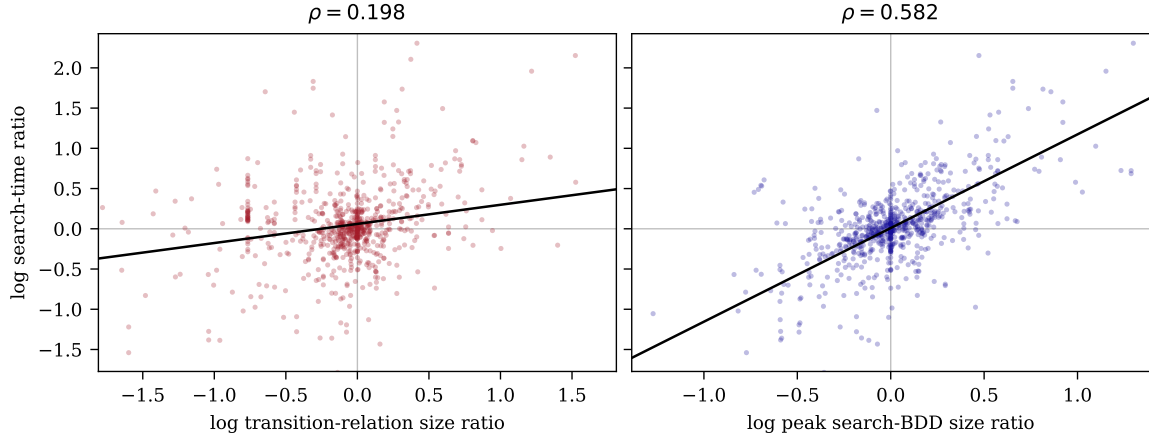


Figure 2: **Why the ordering does not pay off: it moves the wrong quantity.** Each point is one instance on which the method is active and both configurations solve ($n = 759$); axes are log ratios of constraint-aware ordering to the GAMER baseline, with the least-squares trend in black. **Left:** the transition-relation size the objective actually optimizes barely predicts runtime ($\rho = 0.198$) – a diffuse cloud around a nearly flat trend. **Right:** peak search-BDD size, which the ordering leaves essentially unchanged, predicts runtime strongly ($\rho = 0.582$). Shrinking the transition relation is not shrinking the search.

Core Empirical Discovery: Shrinking TR $\neq$ Shrinking Search

Optimizing variable ordering for mutex co-occurrence reliably shrinks Transition Relations (TRs) by 13.3%. However, TR size correlates weakly with search runtime ($\rho = 0.202$). Search runtime is instead strictly governed by peak search-BDD node count ($\rho = 0.593$), which the ordering leaves uncompressed.

4.4 Empirical Ceiling Bounds & Portfolio Selection Limits

We investigated whether algorithm selection, restart schedules, or portfolio wrappers could extract suite-wide gains from constraint-aware variable orderings. We established several theoretical and empirical ceiling bounds:

- **Protocol Resolution Limit:** The selector executes the stock code path whenever it declines to switch, giving 1,076 pairs of runs of *identical* code; their runtimes have $\text{sd}(\log \text{ratio}) = 0.231$, and 33% differ by more than 20%. Two independent full-suite runs of the identical stock configuration differ by 26 instances of coverage. This imposes a strict ± 17 -instance resolution ceiling for suite-wide coverage claims – a bound we confirmed the hard way: an earlier 551-instance sample suggested a +3 suite-wide gain that did not replicate at full scale. Suite-wide coverage differences below this floor should not be trusted regardless of their apparent direction.
- **Oracle Bound:** An ideal 2-ordering oracle selector yields a theoretical coverage gain of +12 instances,

which sits entirely below the ± 17 -instance noise floor. Calibrated models for k -ordering oracles yield +20 ($k = 3$), +24 ($k = 4$), +29 ($k = 6$), and +33 ($k = 8$) instances; practical selectors capturing $\sim \frac{1}{3}$ of oracle value cannot achieve statistically detectable suite-wide wins.

- **Restart Schedules:** In sequential budget-splitting tests under an 1800s limit, only one stock-unsolved instance is rescued by the constraint ordering within 600s (four within 900s), while 50 instances that stock solves require > 900 s. Consequently, all tested budget splits reduce overall coverage.
- **TR-Probe Selection:** A build-time probe selector evaluating initial TR node size selects the correct ordering 66% of the time *when it deviates from stock* (43/65, sign test $p = 0.013$) – a real but weak signal. Measured end-to-end, however, the probe tax dominates: the selector is **18.4% slower** than stock in wall-clock (Table 3), and no configuration in a 150-point parameter sweep is net-positive. Its nominal +5 coverage (1150 vs. 1145) lies well inside the ± 17 -instance noise floor and is not a claim we make.

4.5 Methodological Findings

Our investigation revealed two methodological insights for automated planning evaluations:

4.5.1 Demonstration of Regression to the Mean

Selecting “hard” instances based on baseline runtime artificially skews comparative speedup claims. Filtering to instances where stock SymK takes > 300 s makes constraint ordering appear 20% faster (wall ratio 0.803,

$n = 90$). Conversely, filtering by the constraint ordering’s *own* runtime makes the same method appear 10% slower (1.096, $n = 94$). A symmetric filter – both configurations above the threshold – yields 0.919 ($n = 80$, faster on 54%), and the effect shrinks toward unity as the threshold falls (0.964 vs. 1.025 at > 10 s). Standard asymmetric “hard instance” subsetting produces pure regression-to-the-mean artifacts.

4.5.2 IPC Cross-Year Duplicate Leakage

IPC benchmark instances recur across competition years. Of the 1,847 instances we evaluate, **322 are byte-identical to an instance filed under a different domain** (155 duplicate groups, by MD5 over the problem files). Leave-one-domain-out protocols therefore train and test on literally the same tasks, and per-domain results are not fully independent observations; deduplication should precede any cross-validated claim on this suite. For our own data the hazard is moot because there is little to predict in the first place: ridge regression from preprocessing features (variable count, co-occurrence edge count and density, mutex-BDD and transition-relation sizes) to the ordering’s log runtime effect reaches only $R^2 = 0.06$ under leave-one-domain-out and $R^2 = 0.05$ under leave-one-family-out cross-validation, consistent with Kissmann and Hoffmann’s verdict [5] that ordering quality is not predictable before the BDDs are built.

5. Discussion & Related Work

Our findings touch upon several foundational threads in symbolic classical planning, constraint-graph variable ordering, algorithm selection, and empirical research methodology.

5.1 Symbolic Search and BDD Variable Ordering

Symbolic state-space exploration relies on Binary Decision Diagrams (BDDs) to represent state sets and transition relations compactly. The computational cost of symbolic search is dictated by the BDD node count, which is notoriously sensitive to the chosen variable ordering. GAMER [3] introduced local search over a weighted linear arrangement objective on causal graph edges to optimize BDD variable orders. This scheme was inherited largely unchanged by state-of-the-art planners, including SymBA* [10] and SymK [7, 6].

Kissmann and Hoffmann [5] conducted an extensive study of BDD variable orderings in classical planning, establishing that ordering quality can vary wildly across instances and that finding optimal orderings statically is highly unpredictable. Our diagnostic results reinforce their verdict of unpredictability: while constraint-aware ordering systematically compresses transition relations (TRs), search runtime is ultimately governed by the peak

search-BDD size, which is unaffected by preprocessing-time topology optimizations.

5.2 Invariants and Constraint-Aware Ordering

The inclusion of state invariants, such as h^2 mutexes and exactly-one groups, is essential for pruning spurious states during symbolic search frontier expansion [9]. Torralba and Alcázar [9] observed that these conjoined constraint BDDs depend heavily on the variable ordering and explicitly suggested incorporating mutex edges into the arrangement objective as future work. Our work directly implements and evaluates this open suggestion inside SymK.

Outside classical planning, constraint-graph variable ordering is well-established in SAT and VLSI CAD literature, exemplified by force-directed heuristics like FORCE and MINCE [1]. While approaches like GamerPre [5] incorporate precondition co-occurrence edges, and Torralba et al. [11] use mutexes and invariants for pruning and constraint-BDD compilation while holding the variable order fixed, our framework specifically targets state-invariant mutex and invariant group co-occurrences within the GAMER arrangement objective itself.

5.3 Algorithm Selection, Portfolios, and Methodology

Per-instance algorithm selection and sequential portfolios (e.g., SATzilla [12], Fast Downward Stone Soup [4]) are standard techniques for navigating solver complementary strengths. Our ceiling bound analysis demonstrates why standard selection or restart wrappers over these orderings fail to yield suite-wide gains: the theoretical two-ordering oracle gain (+12 instances) sits below our measured empirical protocol noise floor (± 17 instances).

Furthermore, our methodological findings contribute important cautionary lessons for planning benchmarks:

1. **Regression to the Mean:** We demonstrated that evaluating runtime speedups strictly on “hard” instances filtered by baseline performance introduces severe selection bias, turning neutral performance into an apparent 20% speedup.
2. **Cross-Year Duplicate Leakage:** 322 of the 1,847 evaluated IPC instances are byte-identical to an instance in another domain (155 groups), so domain-wise cross-validation trains and tests on the same tasks; deduplication is a prerequisite for any learned-selector claim on this suite.

Variable ordering was not our first target: we first explored search-control alternatives, including landmark-guided direction selection for bidirectional symbolic search, meet-in-the-middle BDD-overlap signals, and forward-reachability pruning of the backward frontier, all of which we found to be net-neutral or actively harmful, ultimately pointing us toward the variable-ordering lever explored in this paper.

Table 1: Pre-registered confirmatory A/B evaluation results (480 total runs across 2 independent replicates).

Pre-Registered Gate	Target Requirement	Empirical Outcome	Status
Primary: Coverage Contrast	Coverage Gain $\geq +4$	+6 (Rep 1) / +6 (Rep 2)	CONFIRMED
Secondary: Woodworking Speed	Wall Time Geomean ≤ 0.85	0.647 ($1.55\times$ speedup, $n = 94$)	CONFIRMED
Guard: Domain Losses	No Replicated Domain Loss	0 Replicated Domain Losses	PASSED
Integrity: Plan Optimality	0 Cost Mismatches	0 / 480 Cost Mismatches	PASSED

Table 2: Per-domain coverage breakdown on the five signal domains across Replicates 1 and 2. Counts are instances solved within 1800s; $|\mathcal{I}|$ is the number of instances in the domain.

Domain	Weight w_{co}	$ \mathcal{I} $	Stock (R1/R2)	Policy (R1/R2)	Coverage Δ
<i>parking-opt14-strips</i>	1.0	20	1 / 0	3 / 3	+2 / +3
<i>barman-opt11-strips</i>	1.0	20	9 / 9	10 / 11	+1 / +2
<i>tpp</i>	1.0	30	8 / 8	9 / 9	+1 / +1
<i>woodworking-opt08-strips</i>	2.0	30	27 / 28	28 / 28	+1 / +0
<i>woodworking-opt11-strips</i>	2.0	20	19 / 20	20 / 20	+1 / +0
Total Signal Domains	–	120	64 / 65	70 / 71	+6 / +6

Table 3: Suite-wide performance across all 1,847 IPC optimal-STRIPS instances (repaired dataset). Time ratios are **wall-clock relative to stock**, including all selector probe overhead, over instances solved by both the row configuration and stock; intervals are domain-clustered bootstrap 95% CIs. The selector’s search-only ratio, which excludes its probe tax, is 0.992 – its entire deficit is probe overhead.

Configuration	Coverage	Geomean Wall Ratio	Median Ratio	95% Conf. Interval	n
Stock SymK (Baseline)	1145	1.0000	1.0000	–	–
Always-On Constraint Ordering	1139	1.0053	1.0000	[0.97, 1.05]	1126
TR-Probe Selector	1150	1.1837	1.1740	[1.16, 1.21]	1141

6. Conclusion & Future Work

In this paper, we implemented and evaluated constraint-aware variable ordering inside **SymK** across the complete IPC optimal-STRIPS benchmark suite. Our inquiry delivers four primary outcomes:

- **Confirmed Per-Domain Advance:** A pre-registered, double-replicated A/B policy strictly improves stock **SymK** by **+6 coverage** (exactly replicated) across its five target domains and accelerates *woodworking* by $1.55\times$, with zero plan cost mismatches.
- **Bounded Suite-Wide Null:** Enabled universally, the ordering yields a tightly bounded zero effect (coverage 1145 vs. 1139; geomean wall ratio 1.005, 95% CI [0.97, 1.05]), excluding even a 5% suite-wide effect.
- **Mechanism Uncovering:** Diagnostic logging reveals the root cause: the ordering systematically compresses Transition Relation (TR) node sizes by 13.3% ($p \approx 3 \cdot 10^{-21}$), but TR size is decoupled from search cost ($\rho = 0.202$, $R^2 \approx 0.04$), which is instead dictated by peak search-BDD size ($\rho = 0.593$).

- **Ceiling Bounds & Methodological Insights:**

Empirical protocols establish a ± 17 -instance resolution limit, proving why portfolios and selection wrappers cannot yield suite-wide gains, while highlighting critical traps in hard-instance subsetting and IPC duplicate leakage.

6.1 Future Directions

Our diagnostic decoupling points directly toward two promising avenues for future research in symbolic search:

6.1.1 Targeting Peak Search-BDD Size

Because peak search-BDD node count is highly predictive of runtime ($\rho = 0.593$) and bit-deterministic across runs ($r = 0.999$), future variable ordering objectives should target peak search-BDD expansion directly. Since peak BDD sizes are unobservable during preprocessing, developing lightweight online probes or dynamic reordering criteria that estimate search-space width remains an open challenge.

6.1.2 Exploiting Ordering Diversity

Published random-ordering distributions exhibit heavy favorable tails that correlated structural orderings cannot reach [5]. Designing randomized, high-diversity ordering portfolios paired with early-termination rerun controls could unlock the heavy-tail performance gains of symbolic BDD search.

References

- [1] Fadi A. Aloul, Igor L. Markov, and Karem A. Sakallah. FORCE: A fast and easy-to-implement variable-ordering heuristic. In *Proceedings of the 13th ACM Great Lakes Symposium on VLSI (GLSVLSI)*, pages 116–119, 2003.
- [2] Dominik Drexler, Daniel Gnad, Paul Höft, Jendrik Seipp, David Speck, and Simon Ståhlberg. Ragnarok. In *Tenth International Planning Competition (IPC-10): Planner Abstracts, Classical Tracks*, 2023.
- [3] Stefan Edelkamp and Peter Kissmann. GAMER: Bridging planning and general game playing with symbolic search. In *6th International Planning Competition (IPC-6)*, *ICAPS*, 2008.
- [4] Malte Helmert, Gabriele Röger, and Erez Karpas. Fast downward stone soup: A baseline for building planner portfolios. In *ICAPS 2011 Workshop on Planning and Learning (PAL)*, pages 28–35, 2011.
- [5] Peter Kissmann and Jörg Hoffmann. BDD ordering heuristics for classical planning. *Journal of Artificial Intelligence Research (JAIR)*, 51:779–804, 2014.
- [6] David Speck. SymK – a versatile symbolic search planner. In *Tenth International Planning Competition (IPC-10): Planner Abstracts, Classical Tracks*, 2023.
- [7] David Speck, Robert Mattmüller, and Bernhard Nebel. Symbolic top-k planning. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI-20)*, pages 9967–9974, 2020.
- [8] Ayal Taitler, Ron Alford, Joan Espasa, Gregor Behnke, Daniel Fišer, Michael Gimelfarb, Florian Pommerening, Scott Sanner, Enrico Scala, Dominik Schreiber, Javier Segovia-Aguas, and Jendrik Seipp. The 2023 international planning competition. *AI Magazine*, 45(2):280–296, 2024.
- [9] Álvaro Torralba and Vidal Alcázar. Constrained symbolic search: On mutexes, BDD minimization and more. In *Proceedings of the Sixth Annual Symposium on Combinatorial Search (SoCS)*, pages 175–183, 2013.
- [10] Álvaro Torralba, Vidal Alcázar, Daniel Borrajo, Peter Kissmann, and Stefan Edelkamp. SymBA*: A symbolic bidirectional A* planner. In *Eighth International Planning Competition (IPC-8): Planner Abstracts*, pages 105–109, 2014.
- [11] Álvaro Torralba, Vidal Alcázar, Peter Kissmann, and Stefan Edelkamp. Efficient symbolic search for cost-optimal planning. *Artificial Intelligence*, 242:52–79, 2017.
- [12] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research (JAIR)*, 32:565–606, 2008.